Lab 14 oop
Lab Manual: Implementation of the Concepts of Multi-threaded Programming in Java, Implementation of the concepts of Threads Synchronization

1. What is a Thread in Java?

In Java, a **thread** is a lightweight, independent path of execution within a program. Every Java application has at least **one thread** (the main thread), and it can spawn additional threads to perform tasks concurrently.

Why Threads?

Imagine downloading a file, playing music, and scrolling a webpage all happening at once. Threads make this possible without freezing the user interface or overloading a single process.

2. What is Multithreading?

Multithreading is the ability of a program to execute **two or more threads concurrently**, helping perform multiple tasks in parallel without affecting each other.

For example:

- A text editor allows you to type, spell check, and autosave at the same time.
- In a game, rendering graphics, playing sound, and taking user input happen in parallel

Benefits of Multithreading:

- **Improved performance** on multi-core systems.
- **Better resource utilization** (CPU cycles).
- **Non-blocking UI** in GUI applications.
- **Faster execution** in scenarios like file I/O, database operations, etc.

3. Thread Lifecycle (States of a Thread)

A thread in Java passes through several states. Understanding these is critical for managing thread behavior.

State	Description
New	Thread is created but not yet started.

Runnable	After start() is called, thread is ready to run, waiting for CPU time.
Running	Thread is executing its task.
Blocked	Thread is waiting for a monitor lock (used in synchronization).
Waiting	Thread is paused indefinitely until it is notified by another thread.
Timed Waiting	Thread waits for a specified time (e.g., using sleep() or join()).
Terminated	Thread has finished execution.

4. How to Create Threads in Java

A. By Extending the Thread Class

- Override the **run()** method
- Call **start()** to begin execution

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}
```

By Implementing the Runnable Interface

- More flexible (allows multiple inheritance)
- Pass the Runnable object to the **Thread** constructor

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread running via Runnable...");
    }
}
```

5. Important Thread Methods

Method	Purpose
start()	Begins a thread its **run()** method
run()	Contains the logic of the thread
sleep(milliseconds)	Makes thread wait for given milliseconds
join()	Waits for another thread to finish
isAlive()	Returns true if the thread is still running
interrupt()	Interrupts a sleeping or waiting thread

setPriority()	Sets thread priority (from 1 to 10)

6. Thread Priorities

Thread priorities are integers ranging from **1 (MIN_PRIORITY)** to **10 (MAX_PRIORITY)**, with **5 (NORM_PRIORITY)** being the default. However, thread priority is **only a suggestion** to the JVM and not guaranteed.

7. Thread Safety and Synchronization

In multithreaded programs, **multiple threads can access shared resources (like variables, files, databases)**. Without precautions, this leads to problems like **race conditions**.

To prevent this:

- Use **synchronized methods** or **synchronized blocks**
- Ensures only one thread at a time can access the critical section

```
synchronized void withdraw() {
    // Only one thread at a time can withdraw
}
```

8. Daemon Threads

- Daemon threads are **low-priority background threads** that run in the background (e.g., garbage collector).
- JVM exits when only daemon threads are running.

```
t.setDaemon(true); // Must be called before start()
```

Question 1:Simulating a Bank ATM System

Imagine a real-time banking system where multiple users try to withdraw money from the same bank account at the same time. To ensure data consistency and avoid overdrawing, implement a multi-threaded Java program that simulates this scenario using synchronization. Each thread should represent a customer trying to withdraw money. Use proper synchronization so that no two threads can access the account balance simultaneously.

Code:

```
// Bank class representing a shared bank account
```

```java
class Bank {
    private int balance = 1000; // Step 1: Initial account balance set to 1000

    // Step 2: Synchronized method to make withdrawal thread-safe
    public synchronized void withdraw(String user, int amount) {
        System.out.println(user + " is trying to withdraw $" + amount); // Display who is trying to
withdraw
        if (balance >= amount) { // Step 3: Check if balance is sufficient
            System.out.println(user + " is allowed to withdraw."); // Approval message
            try {
                Thread.sleep(1000); // Step 4: Simulate delay in withdrawal (like real ATM processing)
            } catch (InterruptedException e) {
                e.printStackTrace(); // Handle interruption
            }
            balance -= amount; // Step 5: Deduct the amount from the balance
            System.out.println(user + " completed the withdrawal. Remaining balance: $" + balance);
// Print new balance
        } else {
            // Step 6: If balance is insufficient, deny the withdrawal
            System.out.println("Insufficient balance for " + user + ". Withdrawal denied.");
        }
    }
}
// Step 7: Thread class to represent a bank customer
class Customer extends Thread {
    Bank bank;          // Reference to shared bank account
    String userName;    // Customer name
    int amount;         // Withdrawal amount
    // Constructor to initialize the customer
    Customer(Bank b, String name, int amt) {
        bank = b;
        userName = name;
        amount = amt;
    }
    // Step 8: Run method which is executed when thread starts
    public void run() {
        bank.withdraw(userName, amount); // Step 9: Customer tries to withdraw money
    }
}
// Step 10: Main class to run the program
public class BankATM {
    public static void main(String[] args) {
        Bank bank = new Bank(); // Step 11: Create a shared bank object
        // Step 12: Create two customers trying to withdraw money
        Customer c1 = new Customer(bank, "Alice", 700);
        Customer c2 = new Customer(bank, "Bob", 400);
        // Step 13: Start both threads (customers)
        c1.start();
```

```
        c2.start();
    }
}
```

Question 2: Multi-threaded Download Manager Simulation

In a download manager, multiple files can be downloaded at the same time without blocking each other. Simulate a multi-threaded download manager where each file is downloaded in a separate thread. Each thread should show download progress using sleep and print progress messages for simulation.

Code:

```
// Step 1: Thread class that simulates file downloading
class DownloadFile extends Thread {
    private String fileName; // Name of the file to download

    // Step 2: Constructor to initialize the file name
    DownloadFile(String fileName) {
        this.fileName = fileName;
    }
```

```
    // Step 3: Run method executed when thread starts
    public void run() {
        System.out.println("Starting download: " + fileName); // Notify download start
        // Step 4: Simulate 5 chunks of download (20% each)
        for (int i = 1; i <= 5; i++) {
            try {
                Thread.sleep(1000); // Step 5: Simulate time taken to download a chunk
            } catch (InterruptedException e) {
                e.printStackTrace(); // Handle exception
            }
            System.out.println(fileName + " downloaded " + (i * 20) + "%"); // Show download
progress
        }
        System.out.println("Download complete: " + fileName); // Notify that file download is
complete
    }
}
// Step 6: Main class to simulate multiple downloads
public class DownloadManager {
    public static void main(String[] args) {
        // Step 7: Create thread objects for each file
        DownloadFile d1 = new DownloadFile("File_A.mp4");
        DownloadFile d2 = new DownloadFile("File_B.pdf");
        DownloadFile d3 = new DownloadFile("File_C.zip");
```

```
        // Step 8: Start each download in a separate thread
        d1.start();
        d2.start();
        d3.start();
    }
}
```

Question 3 :: Chat Simulation between Two Users

Simulate a chat application where two users can send and receive messages at the same time. Each user will be represented by a thread. One thread will print messages sent by User A and the other will print messages from User B, simulating real-time chat.

Code:

```
// Step 1: Thread class representing a chatting user
class ChatUser extends Thread {
    private String user;          // Name of the user
    private String[] messages;    // Messages to send

    // Step 2: Constructor to set user name and messages
    ChatUser(String user, String[] messages) {
        this.user = user;
        this.messages = messages;
    }
    // Step 3: Run method - prints messages with delay
    public void run() {
        // Step 4: Loop through each message
        for (String msg : messages) {
            System.out.println(user + ": " + msg); // Print message
            try {
                Thread.sleep(1000); // Simulate typing delay
            } catch (InterruptedException e) {
                e.printStackTrace(); // Handle exception
            }
        }
    }
}
// Step 5: Main class to simulate the chat application
public class ChatSimulation {
    public static void main(String[] args) {
        // Step 6: Define messages for both users
        String[] messagesA = {"Hi!", "How are you?", "I'm learning Java Threads."};
        String[] messagesB = {"Hey!", "I'm good!", "That's awesome!"};
        // Step 7: Create thread objects for each user
        ChatUser userA = new ChatUser("User A", messagesA);
```

```
        ChatUser userB = new ChatUser("User B", messagesB);
        // Step 8: Start both threads to simulate chat
        userA.start();
        userB.start();
    }
}
```